PortalOps

The operational event layer for *Portaldot*.

Watch any contract. Get clean, signed, retried, replayable webhooks.

Portaldot Hacker House · Track: Modular Cloud & Middleware

THE PROBLEM

Contracts write down everything they do. *Apps barely see it.*

Raw hex

An on-chain event is just topics and data. You need the contract's ABI to read it.

Reorganisations

The newest blocks can be replaced. Act too early, and you act on something that never happened.

Downtime

Your server goes offline for ten minutes. The chain does not wait. Those events are gone.

So every team rebuilds the same plumbing: polling, decoding, signing, retrying. Usually after the first event they miss.

THE GAP, MEASURED

We pointed PortalOps at the testnet *and just listened*.

32 contracts

225 events

65 kinds

0 readable out of the box

Portaldot V3.0 testnet · blocks 91,528 to 145,105 · measured 2026-10-22 with PortalOps' public explorer (no ABIs registered, so every event is raw)

THE SOLUTION

Watch. Decode. Deliver. *Recover.*



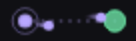
Watch

Follows the chain block by block and spots logs from the contracts you register.



Decode

Turns raw logs into named, typed events using the contract ABI. Stored exactly once.



Deliver

Posts each event to your endpoint as a signed webhook and records what came back.



Recover

Retries with backoff when you are down. Replays any event later, without touching the chain.

`Signed (HMAC-SHA256)` `Retried with backoff` `Replayable, same event id`

WHY PORTALDOT NEEDS IT

A young network. *Event tooling that has to be built.*

What we found

- 32 contracts already emitting 65 kinds of events, none readable without their ABI.
- Builders get raw logs, then write ingestion, retries and signing themselves.
- The EVM layer means standard ABIs work: decoding and webhooks fit naturally.

Built around how Portaldot really behaves

- pallet-revive EVM (V3.0): account mapping handled and documented.
- Only ~256 recent blocks are served by number, so backfill uses log queries.
- Finality comes from the node's finalized head, never guessed: *seen, confirmed, finalized*.
- Weight-based gas: a full contract deploy costs ~180 tPOT, which shaped our demo contracts.

PROOF IT WORKS

Not a mock-up. *Live on the Portaldot testnet.*

PORTALQUEST · DEPLOYED BY US

`0x274F8AbeCAa7FE3772A2cB92F67F2ed4A64bc649`

Block 145,098 · 3 real events: **QuestCreated**, **QuestCompleted**, **RewardIssued**

PORTALPING · DEPLOYED BY US

`0x6a8705e4621dd87a261b1c5f97b73871616537ec`

Block 152,078 · live **Ping** events, captured and delivered on demand

`captured` → `decoded` → `delivered ✓ signature verified` → `fails (500)` → `retried` → `replayed ✓`

170+ automated tests, including against real testnet logs and a real PostgreSQL. A secret scan runs in CI.

THE PRODUCT

Easy at every level, *from click to code to chat*.

Dashboard

A three-step quick start: pick a contract, choose where events go, watch it work.

Playground

No signup. Paste a contract, watch events decode live, send a signed test delivery.

REST API + SDK

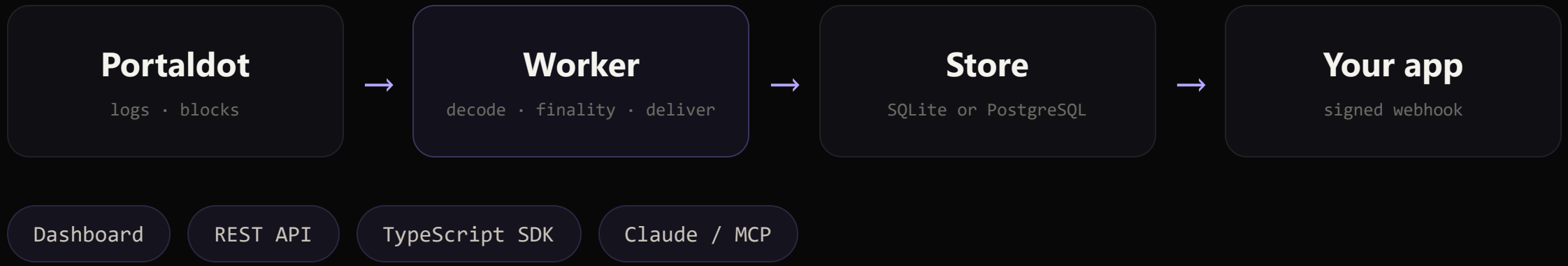
The same API the dashboard uses, plus a typed TypeScript SDK and webhook verification.

Claude

Ten MCP tools: ask in plain English and it registers, wires and tests. Secrets never enter the chat.

ARCHITECTURE

Two small processes. *One database.*



The API and worker share only the database. Chains plug in through a small `ChainAdapter` interface, so new sources are an adapter, not a rewrite.

RELIABILITY & SECURITY

Built to be *trusted*.

Atomic writes

Events, deliveries and the read position move together or not at all.

Signed deliveries

HMAC-SHA256 with a timestamp; verified with constant-time compare.

Honest finality

Seen, confirmed, finalized by the network. Never guessed from depth.

SSRF-safe

Deliveries cannot reach internal addresses, checked when saved and on every send.

Secrets encrypted

AES-256-GCM at rest, shown once. Claude writes them to a local file only.

Reorg-aware

A reorganised log becomes a different event; the old one is marked orphaned.

WHY KEEP SUPPORTING IT

The core is built and live. *Here is what comes next.*

Roadmap

- **Native Rust / ink! adapter** through the existing ChainAdapter interface.
- **More Portaldot networks** and other chains.
- **SDK on npm** and a hosted multi-tenant service.
- **More delivery targets**: chat, email and queues, on the same retry and replay engine.

Why it matters to Portaldot

- Every app on the network needs this layer, and none should have to rebuild it.
- It makes Portaldot contracts usable from ordinary backends, bots and agents.
- Open source, documented, and already running against the live testnet.

PortalOps

Watch any contract. *Never miss an event.*

LIVE APP	portalops.onrender.com
TRY IT, NO SIGNUP	portalops.onrender.com/playground
CODE & DOCS	github.com/nofazechris/portalops

PortalQuest 0x274F8Abe…4bc649 · PortalPing 0x6a8705e4…6537ec